Parcours des écoles d'ingénieurs Polytech (MAM3)

Introduction à la Programmation et à l'Algorithmique

D. Lopez Pacheco



Pour dynamiser et rendre interactif un cours en Cours/TD d'IPA …

POLYTECH
NICE SOPHIA

Java Packages



Un petit cours rapide pour les packages

Nom de packages

- ✓ Les packages Java
  - ❑ sont des groupes de ressources qui fournissent un accès de protection
  - ❑ administration des espaces de noms;
    - il est possible d'avoir 2 classes avec le même nom, si ces classes appartiennent à des packages différents

- ✓ Le mot-clé « *package* » permet de déclarer un package :
  - ❑ Première ligne d'un fichier java
  - ❑ Une seule déclaration par fichier



Quelques définitions utiles

Convention de nommage

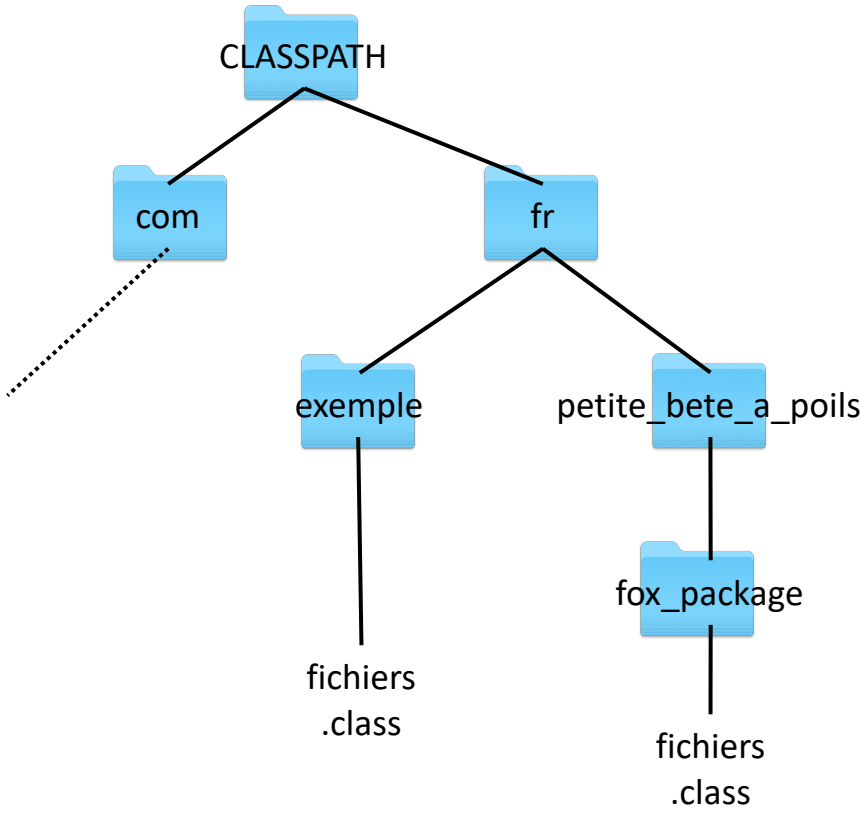
- Les noms des packages s'écrivent en **minuscules** pour les distinguer des noms des classes;
- Les noms de packages adoptent souvent le nom du domaine de l'organisation ou institution qui les fournis
  - Si le nom de l'entreprise possède un mot réservé `java`, le préfixer par « _ »
  - écrire le nom du domaine à l'inverse

- L'entreprise « *petite-bete-a-poils.fr* » fourni le package « *fox-package* »
  - `fr.petite_bete_a_poils.fox_package`



Où enregistrer les packages

- Les packages doivent être enregistrés dans un dossier où vous garderez uniquement vos librairies Java
  - Ajouter ce dossier au CLASSPATH
- Penser aux étiquettes du nom des packages comme des dossiers dans un système de fichiers





Exemples de déclarations de packages




```
//in the Draggable.java file
package graphics;
public interface Draggable {  . . .  }

//in the Graphic.java file
package graphics;
public abstract class Graphic {  . . .}
```

```
//in the Rectangle.java file
package graphics;
public class Rectangle extends Graphic
    implements Draggable {  . . .  }
```

Exemples d'utilisation de packages




```
graphics.Rectangle myRect = new
graphics.Rectangle();
```

```
import graphics.Rectangle;
Rectangle myRectangle = new Rectangle();
```

```
import graphics.*;
Rectangle myRectangle = new Rectangle();
```

```
double r = Math.cos(Math.PI * theta);
```

```
import static java.lang.Math.PI;
// or (to avoid, but works)
import static java.lang.Math.*;
double r = cos(PI * theta);
```

Dossiers et code sources



Code source de « Rectangle.java » du package « graphics »

CLASSPATH/graphics/Rectangle.class

```
//in the Rectangle.java file
package graphics;

public class Rectangle {
    ...
}
```

Package « graphics » fournit par « example.com »

« CLASSPATH/com/example/graphics/Rectangle.class »

```
//in a file using the Rectangle class
import com.example.graphics.Rectangle;
public class AnyClass {
        . . .
}
```

Packages accessibles par défaut



✓ Java définit **deux packages par défaut** (faire référence **sans spécifier** le chemin d'accès) :

- ❑ Le ***package par défaut*** : il s'agit de toutes les classes ne faisant pas partie explicitement d'un package et situées dans les répertoires accessibles (dossier courant ou dans la variable *CLASSPATH*)

- ❑ Le ***package*** **`java.lang`** contenant les classes standards telles que *System, Math, String, etc.*

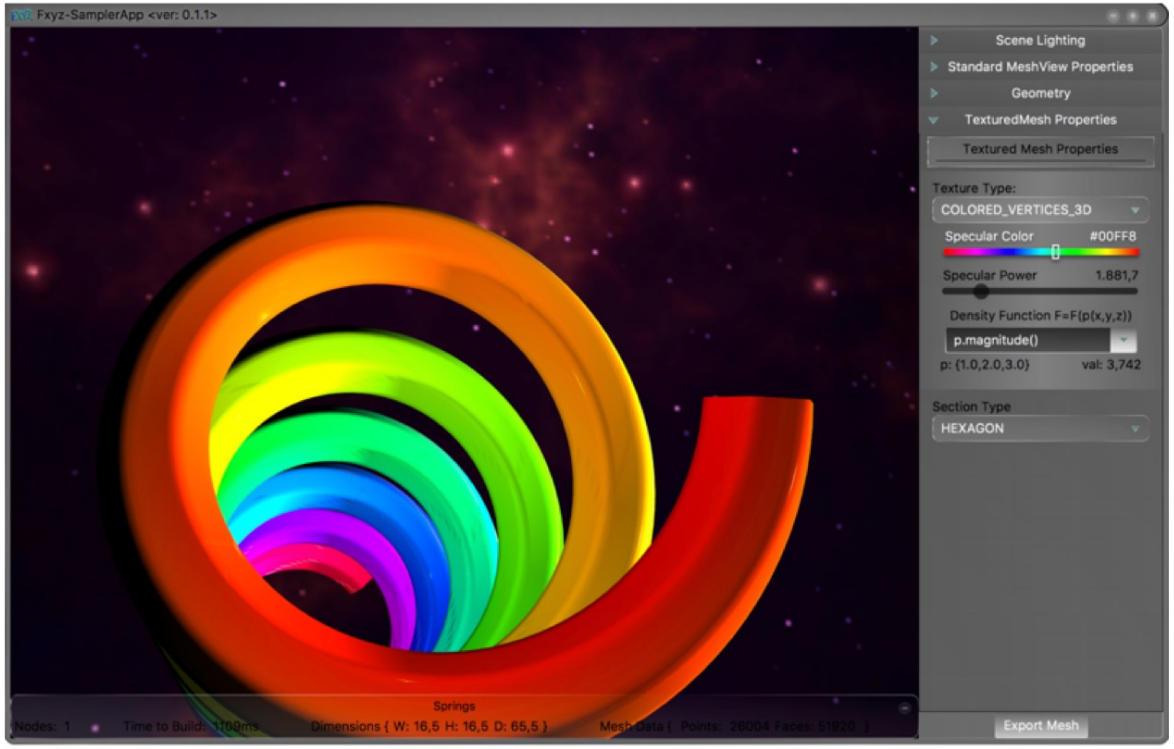
Les fichiers JAR

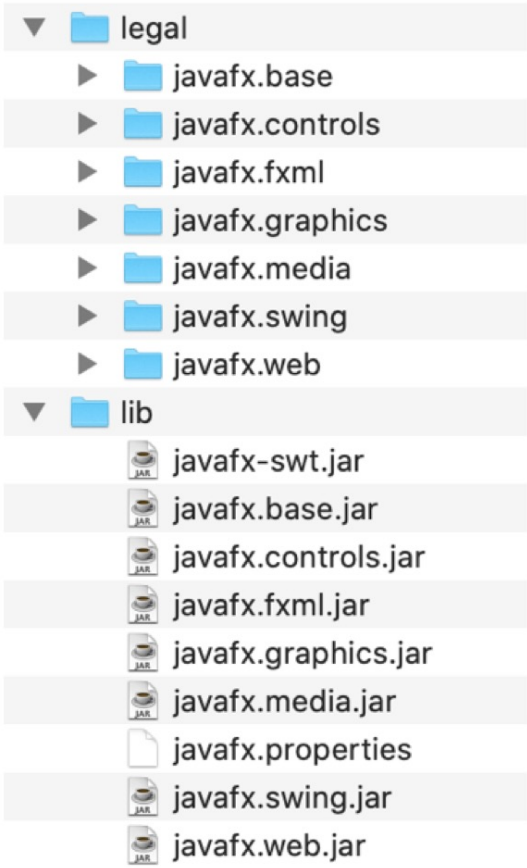
- ✓ Les fichiers JAR (*Java Archive*) sont fréquemment utilisés pour distribuer les logiciels Java
  - ❑ distribution de plusieurs classes dans un seul fichier
  - ❑ les fichiers JAR peuvent également contenir une classe avec `main()`
    - La classe principale
    - JAR exécutable

- ✓ Les fichiers JAR utilisent le format ZIP pour l'**archivage** et **compression de données**.



Exemple : La libraire JavaFX





Contenu d'un fichier JAR

- ✓ 1 dossier META-INF
  - ❑ fichier MANIFEST.MF
    - Méta-données (version d'une librairie, déclaration d'une classe principale, etc.)
- ✓ Les fichiers .class
  - ❑ dans des dossiers, si nécessaire.
  - ❑ Ex. graphics/Rectangle.java

Création/décompression d'un fichiers JAR



✓ La syntaxe de `jar` est proche de celle de `tar` sous Unix :

- ❑ Pour créer l'archive la commande est la suivante :
  - o `jar cvf monArchive.jar dossier_ou_fichiers_class`
- ❑ Pour décompresser l'archive la commande est :
  - o `jar xvf monArchive.jar`





Sauf pour de cas très particulier, on ne doit pas décompresser un fichier JAR pour utiliser la libraire! En effet, nous devons plutôt ajouter les fichiers JAR au CLASSPATH